UNIT IV: Inheritance & Pointers

Section A: Debugging Questions

Question 1: Non-Virtual Destructor Memory Leak in Polymorphic Deletion

Topic: Virtual Functions, Virtual Destructors, Dynamic Dispatch

Corrected Code:

```
#include <iostream>
using namespace std;

class Shape {
public:
    Shape() {
        cout << "Shape created." << endl;
    }

    virtual ~Shape() {
        cout << "Shape destroyed." << endl;
    }

    virtual void render() const {
        cout << "Rendering generic shape." << endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double *vertexArray;
    int vertexCount;

public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];

        cout << "Allocated " << vertexCount
```

```
            << " vertices on heap." << endl;
    }

    ~ComplexPolygon() override {
        cout << "Deallocating polygon vertex heap memory."
            << endl;

        delete[] vertexArray;
    }

    void render() const override {
        cout << "Rendering polygon with "
            << vertexCount << " vertices." << endl;
    }
};

int main() {
    Shape *shapePtr = new ComplexPolygon(1000);

    shapePtr->render();

    delete shapePtr;

    return 0;
}
```

Output:

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Question 2: Inverted Exception Catch Handlers & Unreachable Code

Topic: Exception Handling Mechanism, Catch Handler Ordering

Corrected Code:

```
#include <iostream>
```

```
#include <string>
using namespace std;

class BankingException {
protected:
    string msg;

public:
    BankingException(string m) : msg(m) {}

    virtual string getMessage() const {
        return msg;
    }

    virtual ~BankingException() {}
};

class InsufficientFundsException : public BankingException {
private:
    double shortfall;

public:
    InsufficientFundsException(string m, double sf)
        : BankingException(m), shortfall(sf) {}

    double getShortfall() const {
        return shortfall;
    }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance) {
        throw InsufficientFundsException(
            "Account overdrawn!", amount - balance);
    }

    cout << "Withdrawal successful." << endl;
}
```

```
int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }

    catch (const InsufficientFundsException &ife) {
        cout << "Insufficient Funds: "
             << ife.getMessage() << endl;

        cout << "Shortfall of: $"
             << ife.getShortfall() << endl;
    }

    catch (const BankingException &be) {
        cout << "General Banking Error: "
             << be.getMessage() << endl;
    }

    catch (...) {
        cout << "Unknown banking error occurred." << endl;
    }

    return 0;
}
```

Output: Insufficient Funds: Account overdrawn!

Shortfall of: $250

Question 3: Exception Rethrowing Slicing Bug (`throw e;` vs. `throw;`)

Topic: Throwing, Catching & Rethrowing an Exception, Object Slicing

Corrected Code:

```
#include <iostream>
#include <exception>
#include <string>
using namespace std;

class DatabaseError : public exception {
protected:
    string errorDetails;
```

```
public:
    DatabaseError(string msg) : errorDetails(msg) {}

    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;

    TimeoutError(string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError(
        "Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }

    catch (const DatabaseError &e) {
        cout << "[AUDIT LOG]: Intercepted DB error."
             << endl;

        throw;
    }
}
```

```
int main() {
    try {
        intermediateAuditor();
    }

    catch (const TimeoutError &te) {
        cout << "Client caught TimeoutError! Duration: "
             << te.timeoutMs << " ms" << endl;
    }

    catch (const DatabaseError &de) {
        cout << "Client caught generic DatabaseError: "
             << de.what() << endl;
    }

    return 0;
}
```

Output: [AUDIT LOG]: Intercepted DB error.

Client caught TimeoutError! Duration: 5000 ms

Question 4: Attempting to Instantiate an Incomplete Abstract Class

Topic: Pure Virtual Functions, Abstract Classes, Runtime Interfaces

Corrected Code:

```
#include <iostream>
using namespace std;

class NetworkPacket {
public:
    virtual ~NetworkPacket() {}

    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
```

```
    int frameID;

public:
    VideoPacket(int id) : frameID(id) {}

    void serialize() override {
        cout << "Serializing H.264 Video Frame: "
             << frameID << endl;
    }

    int getPacketSize() override {
        return 1024;
    }
};

int main() {
    VideoPacket packet(1042);

    packet.serialize();

    cout << "Packet Size: "
         << packet.getPacketSize()
         << " bytes" << endl;

    return 0;
}
```

Output:

Serializing H.264 Video Frame: 1042

Packet Size: 1024 bytes

Question 5: Pass-by-Value Object Slicing Suppressing Dynamic Binding

Topic: Runtime Polymorphism, Early vs. Late Binding, Object Slicing

Corrected Code:

```
#include <iostream>
using namespace std;
```

```
class Entity {
public:
    virtual ~Entity() {}

    virtual void attack() const {
        cout << "Entity performs generic scratch."
             << endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        cout << "Dragon breathes scorching fire!"
             << endl;
    }
};

void triggerEncounter(const Entity &e) {
    e.attack();
}

int main() {
    Dragon boss;

    triggerEncounter(boss);

    return 0;
}
```

Output:

```
Dragon breathes scorching fire!
```

Section B: Pseudo-Code Writing Questions

Question 1: Universal Payment Gateway with Runtime Polymorphism

Topic: Pure Virtual Functions, Runtime Polymorphism, Abstract Base Classes

pseudo-code:

```
1. Structured pseudocode
Abstract Base Class: PaymentGateway
ABSTRACT CLASS PaymentGateway
    ABSTRACT METHOD Authorize(amount AS REAL)
        RETURNS BOOLEAN
    ABSTRACT METHOD Capture()
        RETURNS BOOLEAN
    VIRTUAL DESTRUCTOR()
END CLASS
Derived Class: CreditCardGateway
CLASS CreditCardGateway EXTENDS PaymentGateway
    DECLARE cardNumber AS STRING
    DECLARE cvv AS STRING
    CONSTRUCTOR CreditCardGateway(card, securityCode)
        cardNumber <- card
        cvv <- securityCode
    END CONSTRUCTOR
    OVERRIDE METHOD Authorize(amount AS REAL)
        IF LENGTH(cardNumber) = 16 AND LENGTH(cvv) = 3 THEN
            DISPLAY "Credit card authorization successful"
            RETURN TRUE
        ELSE
            DISPLAY "Invalid card details"
            RETURN FALSE
        END IF
    END METHOD
    OVERRIDE METHOD Capture()
        DISPLAY "Credit card payment captured"
        RETURN TRUE
    END METHOD
END CLASS
Derived Class: CryptoGateway
CLASS CryptoGateway EXTENDS PaymentGateway
    DECLARE walletHash AS STRING
    DECLARE gasFee AS REAL
    CONSTRUCTOR CryptoGateway(wallet, fee)
        walletHash <- wallet
        gasFee <- fee
    END CONSTRUCTOR
    OVERRIDE METHOD Authorize(amount AS REAL)
        IF walletHash STARTS WITH "0x" AND gasFee <= amount THEN
```

```
            DISPLAY "Crypto wallet verified"
            DISPLAY "Gas fee validated"
            RETURN TRUE
        ELSE
            DISPLAY "Crypto authorization failed"
            RETURN FALSE
        END IF
    END METHOD
    OVERRIDE METHOD Capture()
        DISPLAY "Cryptocurrency payment captured"
        RETURN TRUE
    END METHOD
END CLASS
Main Program
BEGIN
    DECLARE gatewayPtr AS POINTER TO PaymentGateway
    gatewayPtr <- NEW CryptoGateway(
        "0xABC12345", 25.00)
    DECLARE amount AS REAL <- 1500.00
    DISPLAY "Payment Amount: $", amount
    IF gatewayPtr->Authorize(amount) = TRUE THEN
        IF gatewayPtr->Capture() = TRUE THEN
            DISPLAY "Payment Successful"
        END IF
    ELSE
        DISPLAY "Payment Failed"
    END IF
    DELETE gatewayPtr
END
2. Execution flow
       START
         |
         v
 Create CryptoGateway
         |
         v
 Store address in
 PaymentGateway pointer
         |
         v
 Authorize($1500)
         |
```

```
        v
 Check wallet prefix
        |
        v
 Validate gas fee
        |
        v
 Authorization successful?
      /       \
     Yes        No
      |          |
      v          v
   Capture    Payment Failed
      |
      v
 Payment Successful
      |
      v
 Delete gateway pointer
      |
      v
      END
```

output:

```
Payment Amount: $1500
Crypto wallet verified
Gas fee validated
Cryptocurrency payment captured
Payment Successful
```

Question 2: Resilient Banking Engine with Nested Try-Catch & Rethrowing

Topic: Custom Exception Hierarchy, Nested `try-catch`, Rethrowing

pseudo-code:

```
1. Structured pseudocode
Base Exception Class: TransactionException
CLASS TransactionException
    DECLARE reason AS STRING
    CONSTRUCTOR TransactionException(message)
        reason <- message
    END CONSTRUCTOR
END CLASS
Derived Exception: AccountFrozenException
```

```
CLASS AccountFrozenException EXTENDS TransactionException
    CONSTRUCTOR AccountFrozenException(message)
        BASE TransactionException(message)
    END CONSTRUCTOR
END CLASS
Derived Exception: DailyLimitExceededException
CLASS DailyLimitExceededException EXTENDS TransactionException
    DECLARE excessAmount AS REAL
    CONSTRUCTOR DailyLimitExceededException(message, excess)
        BASE TransactionException(message)
        excessAmount <- excess
    END CONSTRUCTOR
END CLASS
Low-Level Function: DebitLedger()
FUNCTION DebitLedger(amount, accountFrozen, dailyTotal)
    IF accountFrozen = TRUE THEN
        THROW NEW AccountFrozenException(
            "Account is frozen")
    END IF
    IF dailyTotal + amount > 50000 THEN
        excess <- (dailyTotal + amount) - 50000
        THROW NEW DailyLimitExceededException(
            "Daily transaction limit exceeded", excess)
    END IF
    DISPLAY "Debit successful"
END FUNCTION
Intermediate Function: AuditTransactionWrapper()
FUNCTION AuditTransactionWrapper(amount, accountFrozen, dailyTotal)
    TRY
        CALL DebitLedger(amount, accountFrozen, dailyTotal)
    CATCH (e AS TransactionException)
        DISPLAY "SECURITY AUDIT: ", e.reason
        RETHROW
    END TRY
END FUNCTION
Main Program
BEGIN
    DECLARE withdrawal AS REAL <- 1000
    DECLARE accountFrozen AS BOOLEAN <- TRUE
```

```
    DECLARE dailyTotal AS REAL <- 10000
    TRY
        TRY
            CALL AuditTransactionWrapper(
                withdrawal, accountFrozen, dailyTotal)
        CATCH (e AS AccountFrozenException)
            DISPLAY "Client Notice: Account is frozen."
        END TRY
    CATCH (e AS DailyLimitExceededException)
        DISPLAY "Client Notice: Daily limit exceeded."
        DISPLAY "Excess Amount: ", e.excessAmount
    CATCH (e AS TransactionException)
        DISPLAY "Transaction Error: ", e.reason
    END TRY
END
```

output:

```
SECURITY AUDIT: Account is frozen
Client Notice: Account is frozen.
```

Question 3: Polymorphic Game Engine with Virtual Destructors

Topic: Virtual Functions, Virtual Destructors, Polymorphic Object Lifecycles

pseudo-code:

```
1. Structured pseudocode
Abstract Base Class: GameObject
ABSTRACT CLASS GameObject
    ABSTRACT METHOD Update(deltaTime AS REAL)
    ABSTRACT METHOD Render()
    VIRTUAL DESTRUCTOR()
        DISPLAY "GameObject destroyed"
    END DESTRUCTOR
END CLASS
Derived Class: ParticleSystem
CLASS ParticleSystem EXTENDS GameObject
    DECLARE particleCoordinates AS POINTER TO REAL
    DECLARE particleCount AS INTEGER
    CONSTRUCTOR ParticleSystem(count)
        particleCount <- count
        particleCoordinates <- NEW ARRAY OF REAL
                               WITH SIZE count * 2
        DISPLAY "Particle memory allocated"
```

```
    END CONSTRUCTOR
    OVERRIDE METHOD Update(deltaTime)
        DISPLAY "Updating particles"
    END METHOD
    OVERRIDE METHOD Render()
        DISPLAY "Rendering particle system"
    END METHOD
    DESTRUCTOR ParticleSystem()
        DELETE ARRAY particleCoordinates
        DISPLAY "Particle memory released"
        DISPLAY "ParticleSystem destroyed"
    END DESTRUCTOR
END CLASS
Driver Program
BEGIN
    DECLARE sceneEntities AS ARRAY OF POINTER TO GameObject
    sceneEntities[0] <- NEW ParticleSystem(100)
    FOR i <- 0 TO 0
        sceneEntities[i]->Update(0.016)
        sceneEntities[i]->Render()
    END FOR
    FOR i <- 0 TO 0
        DEALLOCATE sceneEntities[i]
    END FOR
END
```

output:

```
Particle memory allocated
Updating particles
Rendering particle system
Particle memory released
ParticleSystem destroyed
GameObject destroyed
```

Question 4: File Stream Pipeline with Stack Unwinding & Catch-All

Topic: Stack Unwinding, Resource Acquisition (RAII), Catch-All Handler

pseudo-code:

```
1. Structured pseudocode
Custom Exception: CRCValidationException
CLASS CRCValidationException
    DECLARE expectedChecksum AS INTEGER
    DECLARE calculatedChecksum AS INTEGER
```

```
    CONSTRUCTOR CRCValidationException(expected, calculated)
        expectedChecksum <- expected
        calculatedChecksum <- calculated
    END CONSTRUCTOR
END CLASS
Class: ResourceGuard
CLASS ResourceGuard
    DECLARE fileHandle AS INTEGER
    CONSTRUCTOR ResourceGuard(fileName)
        fileHandle <- OPEN fileName
        DISPLAY "File resource acquired"
    END CONSTRUCTOR
    DESTRUCTOR ResourceGuard()
        CLOSE fileHandle
        DISPLAY "File resource released"
    END DESTRUCTOR
END CLASS
Function: IngestDataStream()
FUNCTION IngestDataStream(fileName)
    DECLARE guard AS ResourceGuard(fileName)
    DISPLAY "Reading data blocks"
    expectedChecksum <- 500
    calculatedChecksum <- 450
    IF expectedChecksum != calculatedChecksum THEN
        THROW NEW CRCValidationException(
            expectedChecksum, calculatedChecksum)
    END IF
    DISPLAY "CRC validation successful"
END FUNCTION
Main Program
BEGIN
    TRY
        CALL IngestDataStream("mission_data.bin")
        DISPLAY "Data ingestion successful"
    CATCH (e AS CRCValidationException)
        DISPLAY "CRC validation failed"
        DISPLAY "Expected checksum: ", e.expectedChecksum
        DISPLAY "Calculated checksum: ", e.calculatedChecksum
    CATCH (...)
        DISPLAY "Unknown file ingestion error"
    END TRY
    DISPLAY "Program completed"
```

```
END
```

output:

```
File resource acquired
Reading data blocks
File resource released
CRC validation failed
Expected checksum: 500
Calculated checksum: 450
Program completed
```

Question 5: Scientific Telemetry Calibration with `noexcept` Contracts

Topic: Specifying Exceptions, `noexcept` Contracts, Boundary Assertions

pseudo-code:

```
1. Structured pseudocode
Custom Exception: OutOfBoundsException
CLASS OutOfBoundsException
    DECLARE errorMessage AS STRING
    CONSTRUCTOR OutOfBoundsException(message)
        errorMessage <- message
    END CONSTRUCTOR
END CLASS
Function: CalibrateThermalSensors()
FUNCTION CalibrateThermalSensors(rawVoltage)
    NOEXCEPT(FALSE)
    temperature <- rawVoltage * 100
    IF temperature < -273.15 OR temperature > 1500.0 THEN
        THROW NEW OutOfBoundsException(
            "Temperature outside permitted range")
    END IF
    RETURN temperature
END FUNCTION
Function: CommitTelemetry()
FUNCTION CommitTelemetry(finalTemp)
    NOEXCEPT(TRUE)
    TRY
        DISPLAY "Committing telemetry: ", finalTemp
        CALL WRITE_TO_AVIONICS_BUS(finalTemp)
    CATCH (...)
        DISPLAY "Internal telemetry commit error suppressed"
    END TRY
END FUNCTION
Main Program
BEGIN
    DECLARE rawVoltage AS REAL <- -3.0
```

```
    DECLARE calibratedTemp AS REAL
    TRY
        calibratedTemp <- CalibrateThermalSensors(rawVoltage)
        CALL CommitTelemetry(calibratedTemp)
        DISPLAY "Telemetry processing successful"
    CATCH (e AS OutOfBoundsException)
        DISPLAY "Calibration Error: ", e.errorMessage
        DISPLAY "Telemetry rejected"
  CATCH (...)
        DISPLAY "Unknown telemetry error"
    END TRY
    DISPLAY "Safety pipeline completed"
END
```

output:

```
Calibration Error: Temperature outside permitted range

Telemetry rejected

Safety pipeline completed
```